Agentes

Autor: Ismael Sagredo Olivenza

Enunciado

Hay multitud de entornos donde se usa el concepto de Agentes para desarrollar software, pero sus principales aplicaciones tradicionalmente han sido la robótica y los entornos virtuales. Debido a que disponer de robots es algo más complicado a nivel logístico, vamos a utilizar este segundo paradigma como base para realizar nuestra primera práctica de agentes.

En concreto vamos a desarrollar un agente que controle un NPC (Non Plaeyable Character) en un videojuego. Como en todos los demás grados usan el Pacman y debéis estar un poco hartos del mismo entorno, en esta práctica hemos innovado (aunque tampoco mucho) y vamos a utilizar otro videojuego. Eso sí, casi igual de viejuno que Pacman y al que probablemente no habréis jugado nunca :)

El entorno donde va a interactuar vuestro agente es el videojuego **Battle City** que es un videojuego de tanques producido y publicado por Namco como una adaptación del clásico arcade Tank Battalion que es el videojuego original.

En este juego, el jugador debe defender una base (simbolizada por un águila) mientras soporta diferentes oleadas de tanques enemigos. El juego va avanzando por diferentes fases donde hay también diferentes tipos de elementos en el terreno como paredes de ladrillo, paredes metálicas, agua, hierba, etc. Cada tipo de suelo tiene ciertas implicaciones, por ejemplo, el agua no permite pasar a los tanques, pero si los disparos, los bloques de ladrillo son destruibles por los disparos de los tanques, mientras que los metálicos son invulnerables, etc.

Podéis jugar al juego en el siguiente enlace

https://www.retrogames.cz/play_014-NES.php

O en vuestro emulador de referencia de Nes.

En esta ocasión disponemos de una remasterización 3D del juego, donde se moverá nuestro agente. El agente que vamos a programar es un agente de uno de los tanques enemigos y el entorno de pruebas es un subconjunto de las reglas de Battle City muy concretas que definimos a continuación:

- Existen bloques de ladrillos que son rompibles. Los bloques de ladrillos suelen estar agrupados de 4 en 4 o de 2 en 2 pero son rompibles en bloques de 1x1. Por lo que podemos pensar que, aunque parece que el mundo tiene 13x13 casillas, en verdad a nivel lógico tiene 26X26 casillas, es decir 676 casillas en total.
- Los agentes y el jugador pueden estar en cualquier punto del mapa (esto no era así en el juego original debido a las limitaciones de cálculo en coma flotante de la Nes y de las placas arcade primitivas) y no se mueven estrictamente por esas 676 casillas pudiendo estar entre dos de ellas. Esto

Figure 1: Battle City Portada

puede dificultar el movimiento de los agentes en pasillos estrechos.

- Los agentes ocupan algo menos de 2x2 casillas para facilitar los giros y el movimiento entre las casillas (algo que nuevamente no ocurría en el juego original donde estrictamente ocupaba 2x2 casillas)
- Los agentes enemigos tienen 2 puntos de vida y el agente jugador 1 punto de vida.
- El águila está representada por una antena parabólica que simboliza la CommandCenter que debemos defender. En el juego original estaba defendida por una pared más fina, que sólo soportaba 2 toques (los mismos que soporta ahora, pero con una pared visualmente más gruesa)
- Las balas que dispara el tanque si chocan entre si se destruyen.
- Los bloques metálicos son indestructibles y pueden servirte para parapetarte tras ellos de forma segura.
- Existen dos potenciadores, uno que proporciona una vida para el Agente o el jugador (tiene forma de tanque) y otro con forma de estrella que simboliza la salida del nivel. Ambos se instancian en cada ejecución en un punto aleatorio del mapa.
- En el juego original, los agentes enemigos aparecían en oleadas y sabias cuantos tanques enemigos iban a aparecer (había un indicativo de los tanques a la derecha del terreno de combate), en esta versión sólo existen dos agentes, un agente Random programado dentro del juego y el agente que vais a programar vosotros (a parte del jugador) del que os damos una implementación de un agente random como base.
- No puedes disparar mientras no se destruya la bala que has disparado

Vuestro agente aparecerá siempre en la esquina superior izquierda (representado en el diseño del nivel con una miniatura del Agente 47 del videojuego Hitman caricaturizado como un personaje de MegaMan) y el jugador aparecerá siempre junto a la Command Center. El tanque rojo de la figura es el agente aleatorio. El Agente aleatorio es el Base Line, es decir, el agente más simple que podemos crear, por lo que os puede servir de referencia para ver lo buenos que son vuestros agentes... ¿Os vencera el Agente Random? Esperemos que no :)

Vuestro agente se conectará con el juego a traves de un programa en Python. La arquitectura de comunicación está implementada, vosotros sólo os teneis que encargar de implementar el comportamiento del agente.

El juego espera a que se conecte el agente externo antes de empezar. Una vez conectado aparece la palabra Ready y los agentes y el jugador comienzan a moverse por el entorno.

El fichero MainReactiveAgent.py es el punto de inicio del agente que implementa un ReactiveAgent donde tenéis implementado un agente random siguiendo una máquina de estados.

ReactiveAgent inicializa una máquina de estados con un único estado llamado GoToCommandCenter que lejos de la realidad, no llega a la CommandCenter ni de casualidad, ya que se mueve de forma totalmente aleatoria. Pero os doy es

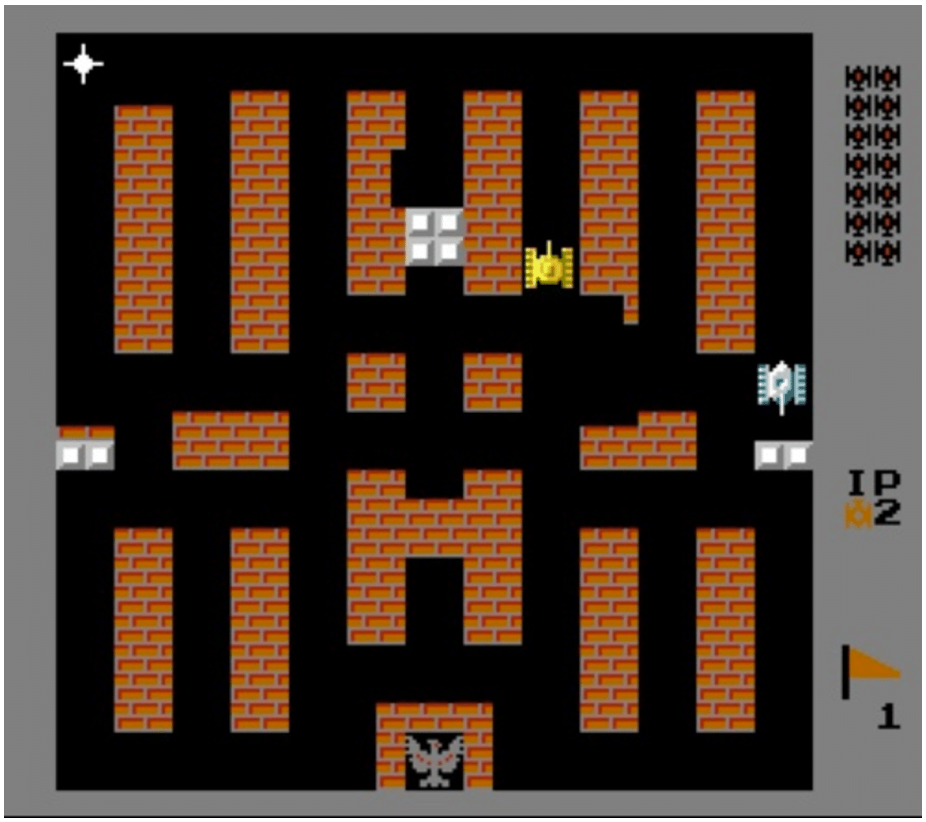

Figure 2: Battle City Captura de pantalla

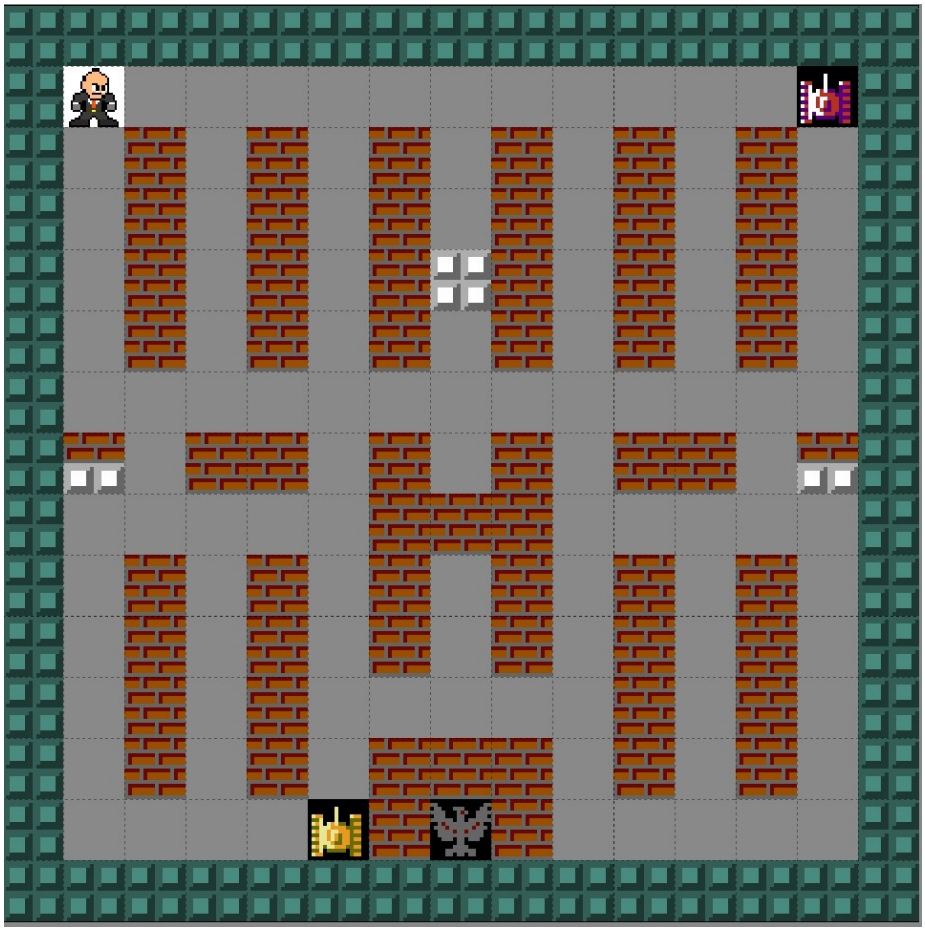

Figure 3: Battle City diseño de la fase

esqueleto de como tenéis que programarlo. En clase os daremos más detalles de como está implementada la maquina de estados.

```
class ReactiveAgent(BaseAgent):
    def __init__(self, id, name):
        super().__init__(id, name)
        dictionary = {
        "GoToCommandCenter" : GoToCommandCenter("GoToCommandCenter")
        }
        self.stateMachine = StateMachine("ReactiveBehavior",dictionary,"GoToCommandCenter")

    #Metodo que se llama al iniciar el agente. No devuelve nada y sirve para contruir el age
    def Start(self):
        print("Inicio del agente ")
        self.stateMachine.Start(self)

    #Metodo que se llama en cada actualización del agente, y se proporciona le vector de per
    #Devuelve la acción u el disparo si o no
    def Update(self, perception, map):
        action, shot = self.stateMachine.Update(perception, map, self)
        return action, shot

    #Metodo que se llama al finalizar el agente, se pasa el estado de terminacion
    def End(self, win):
        super().End(win)
        self.stateMachine.End()
```

Los métodos estan comentados y son auto explicativos pero por si acaso os los describo brevemente.

- Name(): devuelve el nombre del agente que le hayais puesto.
- Id(): Devuelve el identificador, este identificador es el que identifica al agente en el entorno. Por defecto como solo tenemos un agente con "1" es suficiente.
- Start(): este método es invocado una sola vez cuando se inicia el mundo (El agente ya está conectado y ha sido aceptado por el entorno)
- Update(perception,map): Este método es invocado cada cierto tiempo (configurable por linea de comando en el entorno) por el juego para actualizar el estdo de la percepción del mismo. En la percepción teneis una lista con las cosas que ve y sabe el agente en formato float, debeis tratarla como más os interese dentro del método update. Está en formato float para que puedea ser proecesada por algoritmos de Machine Learning aunque en vuestro caso no sea estrictamente necesario. Este método devuelve dos valores, un entero con la acción que quieres realizar y un booleano que indica si debemos disparar o no.
- End(): se os invoca cuando el juego acaba que es por dos motivos, o bien que vuestro agente ha sido destruido (en ese caso win == False) o si la

command center o el jugador han sido destruidos **y habeis llegado a la salida** (en ese caso win == True). No tiene porqué ser el agente el que acabe con la Command Center o con el jugador, podría ser el propio jugador o el otro agente, es indiferente. El caso es que alguien debería ahber hecho el trabajo sucio y tu agente haber sido el mas rápido en llegar a la estrella.

El array de percepción tiene el siguiente esquema donde los números indican la posición del vector y el nombre el campo que representa a nivel lógico:

NEIGHBORHOOD_UP = 0, NEIGHBORHOOD_DOWN = 1, NEIGHBORHOOD_RIGHT = 2, NEIGHBORHOOD_LEFT = 3, NEIGHBORHOOD_DIST_UP = 4, NEIGHBORHOOD_DIST_DOWN = 5, NEIGHBORHOOD_DIST_RIGHT = 6, NEIGHBORHOOD_DIST_LEFT = 7, PLAYER_X = 8, PLAYER_Y = 9, COMMAND_CENTER_X = 10, COMMAND_CENTER_Y = 11, AGENT_X = 12, AGENT_Y = 13, CAN_FIRE = 14, HEALTH = 15, LIFE_X = 16, LIFE_Y=17, EXIT_X=18, EXIT_Y=19, TIME=20

NEIGHBORHOOD_DIST es la distancia del objeto detectado en NEIGHBORHOOD. Los objetos detectados son los siguientes:

NOTHING = 0, UNBREAKABLE = 1, BRICK = 2, COMMAND_CENTER = 3, PLAYER = 4, SHELL = 5, OTHER = 6

El entorno además envía información del mapa y su estad actual, aunque en esta práctica no os servira de mucho, podeis consultarlo si os sirve de algo, pero aquí el agente no puede planificar, por lo que no tiene una especial utilidad.

En el mapa (que se muestra en miniatura en la imagen del juego) pueden aparecer lo siguientes campos:

NOTHING = 0, UNBREAKABLE = 1, BRICK = 2, COMMAND_CENTER = 3, SEMI_BREKABLE = 8, SEMI_UNBREKABLE = 9

Los elementos: PLAYER = 4, SHELL = 5, OTHER_AGENT = 6, LIFE = 7 y EXIT = 10 se consideran dinámicos y no bloqueantes en el mapa (se les peude atravesar) y se mostraran en la percepción pero no en el mapa.

Agente a implementar

El comportamiento del agente que queremos implementar es el siguiente: Nuestro agente debe disparar al jugador si lo tiene a tiro. Lo mismo ocurre con la Command Center que, aunque no está inicialmente visible por el agente (la defiende una pared de ladrillos) el agente conoce en todo momento donde está por lo que puede saber si puede dispararla o no. Si ve una bala que va hacia él, intentará dispararla ya que es más rápido que intentar esquivarla. Para disparar a una bala es muy probable que se requieran varias acciones ya que, por ejemplo, una de ellas podría ser girar hacia la dirección de la bala y otra disparar, pero es posible que no puedas disparar en ese momento. Si encuentra un enemigo, el

agente también debería dispararle. En cuanto a los bloques, queda a vuestro criterio si creéis que es mejor dispararlos o no, así como cualquier otra decisión que os encontréis que debéis tomar. Tambien podéis esquivar una bala si, por ejemplo, no podeis disparar en ese momento. Para ganar el agente debe o bien destruir al jugador o a la Comman Center y una vez destruido uno de los dos, encontrar la salida.

Para saber si un objeto ya no existe en el juego, la percepción marcará su posición con valores negativos en su posición X e Y.

Acciones disponibles:

NOTHING = 0, MOVE_UP = 1, MOVE_DOWN = 2, MOVE_RIGHT = 3, MOVE_LEFT = 4

No se puede mover en diagonal, si se intenta el agente no se moverá.

Disparo si o no.

Se puede disparar y moverse simultáneamente. Si se intenta disparar, pero el agente no puede disparar ignorará la orden.

Un agente será mejor que otro si es capaz de vencer en menor tiempo.

Se pide:

En un pdf, jupiter notebook o donde os venga mejor hacerlo:

1. Descripción detallada del entorno usando REAS.
2. Descripción de qué tipo de entorno se trata: estocástico, discreto, episódico.... justificando las respuestas.
3. Qué tipo de agente es y explícalo.
4. Diseño preliminar del agente como una máquina de estados (la próxima clase lo chequearemos)
5. Diseño final del agente despues de haberlo implementado.
6. Qué diferencias hay entre el agente inicialmente diseñado y el final y porqué.
7. Qué es lo que más os ha costado resolver y a qué creeis que es debido: a la naturaleza del agente o a la naturaliza del entorno (o a ambas cosas). Explícalo con tus palabras.

En python Implementación del agente.

Test de prueba. Vamos a realizar 3 test que podéis recrear vosotros mismos en vuestras pruebas.

1. Que el agente venza sin que el jugador haga nada. El mejor agente deberá resolver esto en el menor tiempo posible.
2. Que el agente venza sin que el jugador dispare. El mejor agente deberá resolver esto en el menor tiempo posible.
3. Que el agente tenga una inteligencia aparente compitiendo contra el jugador. Este último punto es más subjetivo, pero debería ser lo suficientemente

inteligente como para que matarlo no sea algo muy simple. Obviamente no esperamos conseguir un agente que venza al humano.

Cosas a valorar

- Preguntas de teoría a entregar: 15% de la nota
- Que el agente cumpla con las dos primeros test. Red line para superar la práctica y cuenta un 40% de la nota.
- Que en el tercer test su comportamiento sea aparentemente inteligente: 30% de la nota.
- Código razonablemente bien desarrollado con buenas prácticas de programación, eficiente y minimamente robusto frente a fallos: 15%

NOTA: si se detecta un bug que impida el desarrollo del agente, enviarme un correo a isagredo@ucm.es

Comandos para manejar el entorno:

- Tecla M (mientras esté el player vivo) apaga el sonido
- Podeis invocar por linea de comando el entorno con los siguientes parametros:

BattleCity.exe -level 1 -snapshot 0.25

-level puede valer 1 o 2, por defecto es 1. El agente debería poder superar ambos niveles.

-snapshot nos determina cada cuanto tiempo informo al agente de la percepción. Por defecto está a 0.2 pero puedes modificarlo (Se recomienda no bajarlo demasaido para no saturar la CPU)

-help on: muestra mas comandos aunque algunos no son utiles para la práctica actual.

Opcional

- Implementar el agente con una máquian de estados jerarquica (la ma´quina de base está preparado para ello) (1 chocopunto)
- Implmentar el agente usando un Behavior Tree (2 chocopuntos)